Emanuel Charro, 862158963
JoseAngel Pulido, 862398468

**Introduction:**

Computer vision focuses on enabling computers to interpret visual information from images and video. Many modern systems use cameras and image processing algorithms to detect objects, recognize patterns, and track motion in real time. One interesting application of computer vision is gesture-based interaction, where hand movements are used to control computer systems.

This project explores the development of a virtual air mouse using computer vision techniques. The goal is to control the mouse cursor using hand gestures detected by a webcam instead of using a physical mouse. The position of the index finger is used to move the cursor, while certain finger gestures are interpreted as mouse clicks.

The system was implemented using Python and MediaPipe, which provides real-time hand landmark detection. MediaPipe detects important points on the hand, called hand landmarks, which represent the position of fingers and joints. These landmarks can be used to analyze the structure and motion of the hand.

A gesture recognition model was trained to classify three gestures used in the systems: point, left-click, and right-click. By analyzing the spatial arrangement of the hand landmarks, the model can recognize the gesture being performed

Another important part of the computer vision systems is the ability to process images in real time. For gesture-based interfaces, the system must detect hand movements and respond quickly without noticeable delay. By analyzing each frame from the webcam and tracking hand landmarks, the system can recognize gestures and translate them into mouse actions. This shows how computer vision techniques can be used to create interactive systems based on natural hand movements.

The objective of this project is to apply computer vision techniques learned in class, such as feature extraction, pattern recognition, and image-based classification, to build a working gesture-controlled mouse system.

**Related work:**

Computer vision systems rely heavily on extracting meaningful information from images. From the textbook by Burger and Burge in Digital Image Processing an algorithmic introduction, image analysis often begins by detecting important features within an image, such as edges, shapes, or key points. These features can then be used to recognize objects or patterns with the scene.

In gesture recognition systems, the human hand is treated as an object whose structure can be analyzed through feature extraction. Important features such as fingertip positions, finger angles and joint locations can be used to describe the configuration of the hand. Once these features are extracted, pattern recognition methods can be used to classify the gesture being performed

Modern frameworks such as MediaPipe, provide efficient hand tracking models that detect 21 hand landmark points from an image. These landmarks represent key points on the hand, including fingertips and joints. By analyzing the relative position of these landmarks, it is possible to identify hand gestures in real time.
Previous work has demonstrated that computer vision techniques can be used to implement gesture-controlled interfaces. For example, several virtual mouse systems use fingertip tracking to control cursor movement and detect pinch gestures for mouse clicks. These systems typically follow a pipeline that includes image capture, feature extraction, gesture classification, and action mapping.

The approach used in this project follows similar computer vision principles. Hand landmarks are extracted from webcam images, a gesture classification model is trained using labeled data, and the predicted gestures are mapped to mouse control actions

**Technical approach**

**Implementation using Python:**

The system was implemented in Python using Google Colab and several computer vision libraries. The main libraries used in the code include MediaPipe, TensorFlow, NumPy, and OpenCV. These libraries were used to load the dataset, train the gesture recognition model, and evaluate its performance.

The first part of the code installs the required dependencies and downloads the MediaPipe hand landmark model. After this setup, the dataset containing the label gesture images is loaded from Google Drive. The dataset folders represent the gesture classes used for the system, which include point, left-click, and right-click.

Once the dataset is loaded, the code extracts hand landmark features from each image using the MediaPipe gesture recognizer tools. These landmark features represent the positions of the fingers and joints and are used as input data for training the gesture classification model. The dataset is then divided into training, validation, and test sets, which allows the model to be trained and evaluated properly.

During the training stage, the model learns patterns in the landmark features that correspond to each gesture. The model is trained for several epochs and then exported as a gesture_recognizer.task file. This exported model can later be used to recognize gestures in real time.

**Model Evaluation and Metrics:**

After the model is trained, the code evaluates its performance using the test dataset. Several classification metrics are computed to measure how well the model

recognizes gestures. These metrics include precision, recall, F1-score, confusion matrix, and ROC curves.

The confusion matrix helps visualize how many gestures were correctly or incorrectly classified. Precision and recall provide additional information about the model's prediction accuracy for each gesture class. ROC curves are also generated to analyze how well the model separates the gesture categories.

These evaluation results help determine whether the trained model performs well enough to be used for the virtual mouse application.

**Virtual Mouse Implementation:**

Once the gesture recognition model is trained and evaluated, it can be used to implement the virtual mouse system. The virtual mouse program captures frames from the webcam and processes each frame using the MediaPipe hand tracking model.

The system detects the hand landmarks in each frame and uses the trained gesture model to classify the gesture being performed. The index fingertip landmark is used to determine the cursor position on the screen. The coordinates of the fingertip are mapped from the camera to the screen coordinates so that the cursor moves according to the finger movement.

When a specific gesture is detected, the system performs the corresponding mouse action. For example, a point gesture controls cursor movement, while other gestures trigger left-click or right-click actions. This allows the user to interact with the computer using hand movements instead of a physical mouse.

**Team Contribution**

This project was completed as a team effort. One team member focused on developing the gesture recognition model and evaluation metrics, including training the model, analyzing classification performance and generating the confusion matrix and ROC curve results. The other team member focused on implementing the virtual mouse system, which involved integrating the trained model with real-time hand tracking and mapping the detected gestures to mouse control actions. By combining the two components, the final system was able to recognize the hand gestures and convert them into real-time cursor control.

**Experiment results(data & results)**

The gesture recognition system was evaluated using a dataset of hand gesture images representing the gestures required to control the virtual mouse. The dataset contained approximately seven hundred images of hand gestures that were collected and

organized into three categories: point, left-click, and right-click. These gesture classes correspond directly to the mouse actions implemented in the system. The point gesture is used to control cursor movement across the screen, while the left-click and right-click gestures simulate the corresponding mouse buttons. Organizing the dataset into these gesture classes allowed the model to learn the visual differences between the hand configurations associated with each mouse action.

Before training the gesture recognition model, each image in the dataset was processed using the MediaPipe hand tracking framework. MediaPipe detects twenty-one hand landmark points that represent the positions of important joints and fingertips of the hand. These landmarks include points for each finger joint as well as the base of the palm. Instead of using the entire image as input to the classifier, the system uses the landmark coordinates as the main features for gesture recognition. These coordinates describe the spatial configuration of the hand and allow the system to analyze the relative positions of the fingers.

The landmark coordinates are normalized values between zero and one. Normalizing the coordinates makes the system less sensitive to variations in hand size or distance from the camera. This means the gesture recognition model can still perform accurately even if the user moves their hand closer to or farther from the webcam. This normalization step is an important part of the preprocessing stage because it improves the consistency of the feature representation used by the model.

After the preprocessing stage, the dataset was divided into three subsets so that the model could be trained and evaluated properly. Approximately eighty percent of the images were used as training data, while the remaining images were split between validation and testing datasets. The training dataset allows the model to learn the patterns of hand landmark positions associated with each gesture class. The validation dataset helps monitor the training process and prevents the model from overfitting the training data. The testing dataset is used after training to evaluate the final performance of the model on new images that were not used during training.

The gesture recognition model was trained using MediaPipe Model Maker in Python. During training, the model analyzes the landmark coordinates extracted from each image and learns to associate these patterns with the correct gesture labels. The training process runs for several epochs so that the model gradually improves its classification accuracy. Once the training stage is complete, the trained model is exported as a gesture recognizer file that can be used for gesture prediction during the virtual mouse operation.

To evaluate the performance of the trained model, several classification metrics were calculated. These metrics include precision, recall, F1-score, confusion matrix analysis, and ROC curves. Each of these metrics provides useful information about how well the model performs when recognizing hand gestures. Precision measures how many predicted gestures were correct, while recall measures how many of the actual gestures were successfully detected by the model. The F1-score combines precision and recall into a single value that summarizes the overall classification performance.

The classification report generated from the test dataset shows that the gesture recognition model performs very well for the gesture classes used in the virtual mouse system. The point gesture achieved a precision value of 1.00 and a recall value of 1.00, which means that all point gestures in the test dataset were correctly classified by the model. The left-click gesture achieved a precision of 0.95 and a recall of 1.00, indicating that almost all predictions were correct and that the model was able to detect all left-click gestures in the dataset. The right-click gesture achieved a precision of 1.00 and a recall of 0.95, showing similarly strong performance with only a small number of misclassified samples.

The overall accuracy of the gesture recognition model was approximately ninety-eight percent. This high accuracy indicates that the trained model is capable of reliably distinguishing between the different hand gestures used in the system. Achieving this level of accuracy demonstrates that the landmark-based feature representation is effective for gesture classification tasks.

A confusion matrix was also generated to visualize the classification results. The confusion matrix compares the true gesture labels with the predicted labels generated by the model. Most of the predictions appear along the diagonal of the matrix, which represents correct classifications. For example, twenty-six point gestures were correctly classified as point gestures. Similarly, nineteen left-click gestures and twenty right-click gestures were correctly classified. Only a small number of gestures were misclassified, primarily between the left-click and right-click categories. This suggests that the model can accurately distinguish between most gesture types, although some gestures with similar hand configurations may occasionally be confused.

To further analyze the performance of the model, ROC curves were generated using a one-versus-rest classification approach. The ROC curve shows the relationship between the true positive rate and the false positive rate for each gesture class. The area under the ROC curve, also known as the AUC value, provides a measure of how well the model can separate the gesture classes. In this project, the AUC values for the

gesture classes were close to 1.0, which indicates strong classification performance and clear separation between the different gesture categories.

The experimental results also include visual examples showing the detected hand landmarks and the predicted gesture labels. In these images, the MediaPipe framework overlays a hand skeleton on top of the original image to illustrate the detected landmark positions. These landmarks make it easier to see how the system interprets the hand structure and determines the gesture being performed. Each example image displays both the true gesture label and the predicted label generated by the model. In most cases, the predicted labels match the true labels, which confirms the accuracy of the gesture recognition system.

These visual results also demonstrate that the hand landmark detection stage works reliably. The MediaPipe framework consistently detects the key points of the hand even when the hand is slightly rotated or positioned differently relative to the camera. This robustness is important for real-time applications because users may move their hands in different ways while interacting with the system.

Overall, the experimental results show that the gesture recognition system performs reliably for controlling the virtual mouse. The combination of hand landmark detection, gesture classification, and coordinate mapping allows the system to convert hand gestures into mouse actions with high accuracy. The results demonstrate that computer vision techniques such as feature extraction, landmark detection, and gesture classification can be successfully applied to create a practical gesture-based human–computer interaction system.

**Conclusion:**

This project applied computer vision techniques to develop a gesture-controlled virtual mouse system. Using a webcam and MediaPipe hand tracking, the system is able to detect hand landmarks and classify gestures in real time.

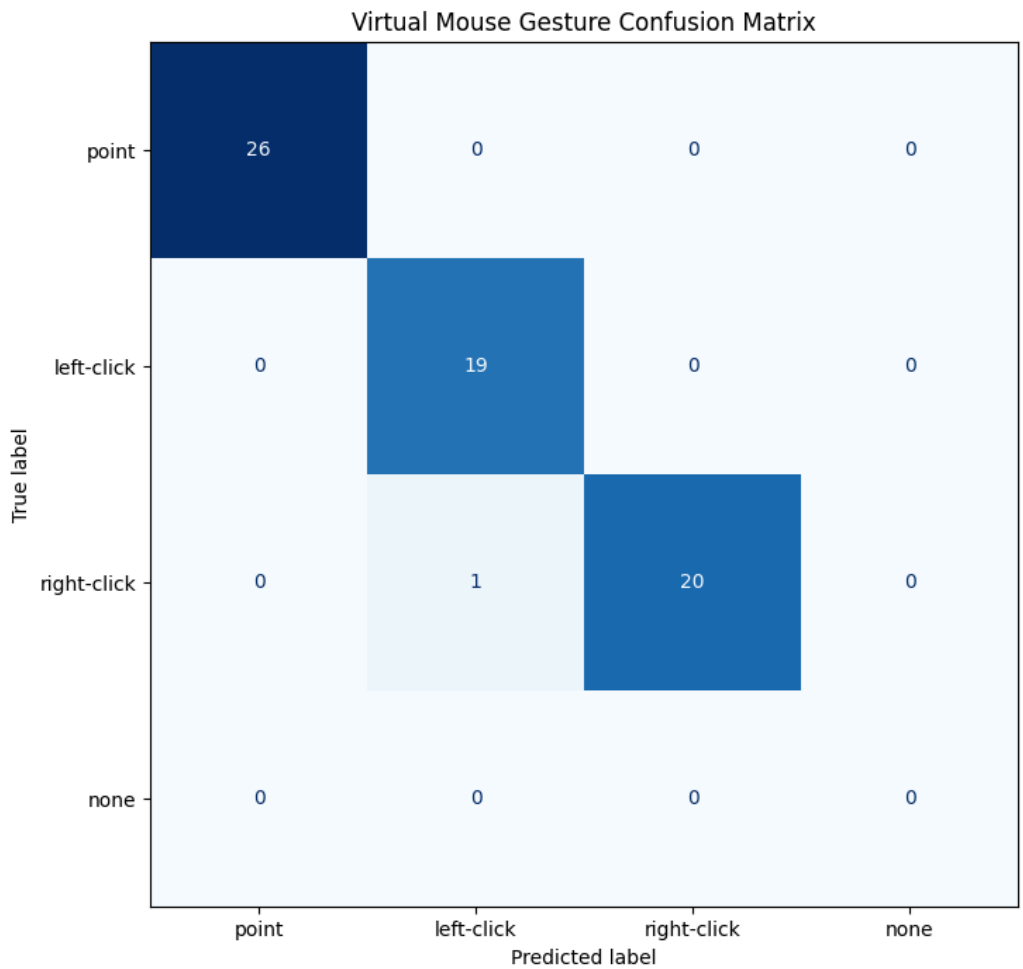
The trained gesture recognition model achieved high classification accuracy and was able to correctly identify the defined gestures used for cursor movement and mouse clicks. Evaluation metrics such as confusion matrices and ROC curves showed that the model performs well in distinguishing between gesture classes.

Although the system worked reliably in most cases, factors such as lighting conditions and background complexity can still affect detection accuracy. Future improvements could include collecting a larger dataset, adding more gesture types, and improving robustness in different environments.

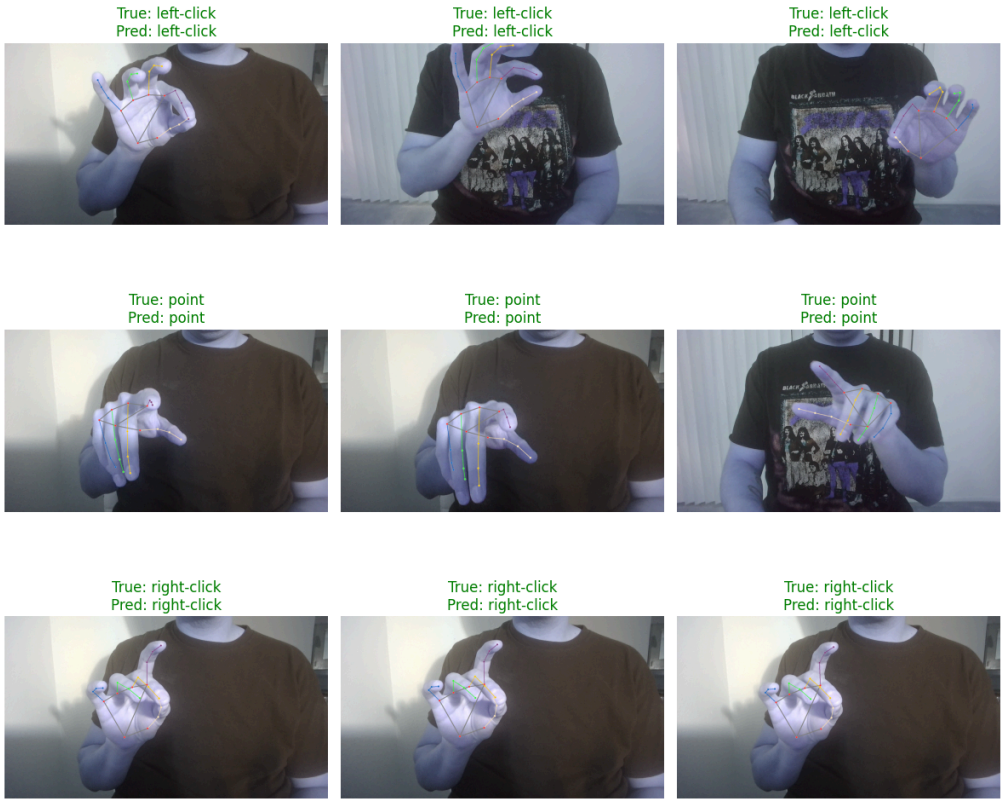
Overall, the project demonstrates how computer vision techniques such as feature extraction, gesture classification, and real-time image processing can be applied to create practical human–computer interaction systems.

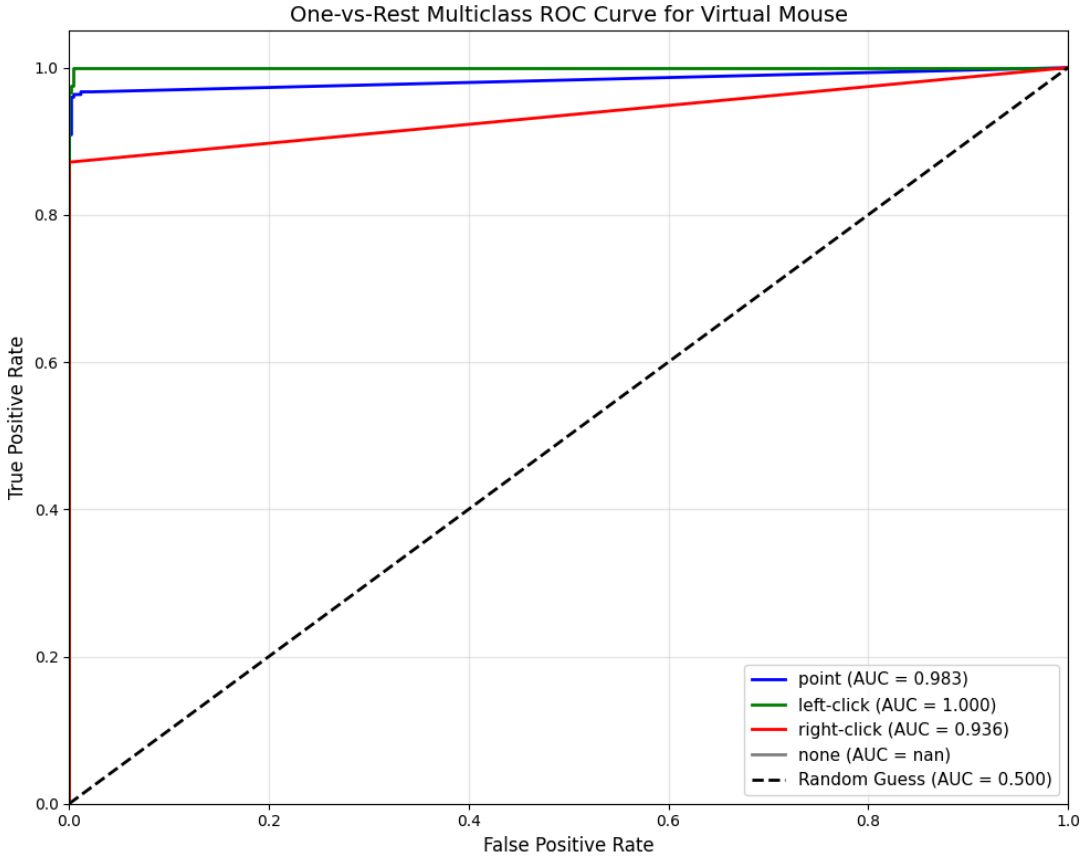
## Appendix

```
==============================================================
Model Classification Summary
==============================================================
              precision    recall  f1-score   support

       point       1.00      1.00      1.00        26
  left-click       0.95      1.00      0.97        19
 right-click       1.00      0.95      0.98        21
        none       0.00      0.00      0.00         0

    accuracy                           0.98        66
   macro avg       0.74      0.74      0.74        66
weighted avg       0.99      0.98      0.98        66
```



Original Images & Extracted Skeletons



**References:**


- Code, Linux. “Real-Time AI Virtual Mouse System Using Computer Vision (AI Virtual Mouse).” Thelinuxcode.com, 4 Feb. 2026, thelinuxcode.com/real-time-ai-virtual-mouse-system-using-computer-vision-ai-virtual-mouse/.
- Pundir, Arnav. “Air-Mouse.” GitHub, 19 May 2025, github.com/ArnavPundir22/Air-Mouse.
- GeeksforGeeks. “RealTime AI Virtual Mouse System Using Computer Vision.” GeeksforGeeks, 30 Nov. 2023, www.geeksforgeeks.org/computer-vision/ai-virtual-mouse/.
- Jainth, Shalini. “Computer Vision Based Real Time Air Mouse Using Webcam.” Ijrar.org, International Journal of Research and Analytical Reviews, Apr. 2022, ijrar.org/papers/IJRAR22B1498.pdf.